# Practical 15: CRUD Operations in a Full-Stack Web Application

## Aim

To study and implement Create, Read, Update, and Delete (CRUD) operations on a database using Python and SQLite.

## Theory

CRUD refers to the four fundamental operations performed on persistent data in virtually any data-driven application: Create (insert new records into the database, corresponding to the SQL INSERT statement and typically an HTTP POST request in a web API), Read (retrieve one or more existing records, corresponding to SQL SELECT and an HTTP GET request), Update (modify the values of existing records, corresponding to SQL UPDATE and an HTTP PUT/PATCH request), and Delete (remove records, corresponding to SQL DELETE and an HTTP DELETE request). These four operations form the backbone of full-stack web applications: the frontend (UI) captures user actions, the backend (server-side application logic, e.g., Flask or Express) receives requests and translates them into the appropriate database operations, and the database (here, SQLite) persists the data using SQL statements executed through a database cursor, with every write operation (Create/Update/Delete) requiring an explicit commit() to make the change durable. Understanding CRUD is essential because nearly every real-world feature - user registration, a shopping cart, a content management system - is ultimately composed of these same four operations applied to different types of data.

## Algorithm

1. Connect to an SQLite in-memory database and create a products table.
2. Create: insert new product records and commit the transaction, then display all records.
3. Read: query and display a specific product record by its id.
4. Update: modify the price of a specific product and commit, then display all records.
5. Delete: remove a specific product record and commit, then display the remaining records.
6. Close the database connection.

## Program

```
import sqlite3

conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT, price INTEGER)")

# Create
cur.execute("INSERT INTO products (name, price) VALUES ('Pen', 10)")
cur.execute("INSERT INTO products (name, price) VALUES ('Book', 50)")
conn.commit()
print("After CREATE:")
for row in cur.execute("SELECT * FROM products"):
    print(row)

# Read
print("\nREAD product id=1:")
print(cur.execute("SELECT * FROM products WHERE id=1").fetchone())

# Update
cur.execute("UPDATE products SET price=15 WHERE name='Pen'")
conn.commit()
print("\nAfter UPDATE:")
for row in cur.execute("SELECT * FROM products"):
    print(row)
# Delete
```

```
cur.execute("DELETE FROM products WHERE name='Book'")
conn.commit()
print("\nAfter DELETE:")
for row in cur.execute("SELECT * FROM products"):
    print(row)

conn.close()
```

## Output

```
After CREATE:
(1, 'Pen', 10)
(2, 'Book', 50)

READ product id=1:
(1, 'Pen', 10)

After UPDATE:
(1, 'Pen', 15)
(2, 'Book', 50)

After DELETE:
(1, 'Pen', 15)
```

## Result / Conclusion

CRUD operations (Create, Read, Update, Delete) were successfully implemented and verified on an SQLite database, forming the core of a full-stack web application's backend logic.

---

## Viva Questions

### Q1. What do CRUD operations map to in SQL and HTTP?

**Ans:** Create → SQL INSERT, HTTP POST. Read → SQL SELECT, HTTP GET. Update → SQL UPDATE, HTTP PUT (full replace) or PATCH (partial update). Delete → SQL DELETE, HTTP DELETE. This mapping is the foundation of RESTful API design: each resource (e.g., /products) exposes these four operations via the corresponding HTTP methods on its URL.

### Q2. What is the difference between SQL UPDATE and DELETE?

**Ans:** UPDATE modifies the values of one or more columns in existing rows that match a WHERE condition — the rows remain in the table. DELETE removes entire rows that match a WHERE condition. Both require a WHERE clause in practice (omitting it affects ALL rows). Both are write operations requiring conn.commit() to persist. TRUNCATE (not in SQLite standard) removes all rows faster by dropping and re-creating the table internally.

### Q3. What happens if you run UPDATE or DELETE without a WHERE clause?

**Ans:** Without a WHERE clause, UPDATE modifies every row in the table and DELETE removes every row. This is usually a catastrophic mistake in production. Always include a WHERE clause to target specific records (e.g., WHERE id=1). In production databases, use transactions so you can rollback if you accidentally run a destructive command without the WHERE clause.

### Q4. What is a PRIMARY KEY and why is it important in CRUD?

**Ans:** A PRIMARY KEY is a column (or combination of columns) that uniquely identifies each row in a table. It must be NOT NULL and UNIQUE. In SQLite, INTEGER PRIMARY KEY auto-increments. In CRUD: the primary key is the target for Read (WHERE id=?), Update (SET ... WHERE id=?), and Delete (WHERE id=?) operations. Without a primary key, you cannot uniquely address individual records for update or deletion.

### Q5. What is the difference between a full-stack and a backend-only application?

**Ans:** A backend-only application exposes data via APIs (JSON responses) but has no built-in user interface — clients (mobile apps, SPAs) consume the API separately. A full-stack application includes both the frontend (HTML/CSS/JS user interface, e.g., Jinja2 templates in Flask) and the backend (server logic, database) in one codebase. Full-stack covers the entire application from database to UI; backend-only is one tier of a multi-tier

architecture.